SPN 密码差分攻击实验报告

一、实验目的

1. 理解 SPN（代换-置换网络）分组密码的结构和原理
2. 掌握差分密码分析的基本原理和方法
3. 通过 C++编程实现 SPN 加密算法和差分攻击
4. 分析差分攻击的有效性和样本数量对攻击成功率的影响

二、实验原理

2.1 SPN 密码体制

SPN（Substitution-Permutation Network）是一种经典的分组密码结构，由多轮代换和置换组成。本实验实现的 SPN 参数如下：

- **分组长度**：16 位
- **S 盒数量**：4 个（每个 4×4）
- **轮数**：4 轮
- **密钥长度**：16 位×5 = 80 位（实际使用 16 位轮密钥）

S 盒定义

```
输入:  0 1 2 3 4 5 6 7 8 9 A B C D E F
输出:  E 4 D 1 2 F B 8 3 A 6 C 5 9 0 7
```

P 盒置换

P 盒将 16 位进行位置置换，置换规则为：

```
输出位 i ← 输入位 PBOX[i]
PBOX = {0,4,8,12,1,5,9,13,2,6,10,14,3,7,11,15}
```

加密流程

```
第 1-3 轮: state = Pbox(Sbox(state ⊕ Kᵢ))
第 4 轮:   state = Sbox(state ⊕ K₄) ⊕ K₅
```

2.2 差分密码分析原理

差分密码分析的基本思想是通过分析明文对差值与密文对差值之间的相关性来恢复密钥。

核心概念：

- **输入差分**：$\Delta X = X \oplus X^*$
- **输出差分**：$\Delta Y = Y \oplus Y^*$
- **差分特征**：通过 S 盒和 P 盒传播的差分路径

S 盒差分分布：对于 4 位 S 盒，输入差分α到输出差分β的计数满足：

- 对于随机 S 盒，平均计数为 16/256 = 0.0625
- 高概率差分会显著偏离平均值

本实验利用的差分特征：

- 输入差分：$\Delta X = 0xB00$（即第 1 个 nibble 为 0xB，其余为 0）
- 目标差分：$\Delta U_2 = \Delta U_4 = 0x6$

三、程序实现

3.1 核心数据结构

```
// S 盒及其逆
constexpr array<uint8_t, 16> SBOX = {...};
array<uint8_t, 16> SBOX_INV;

// P 盒及其逆
constexpr array<int, 16> PBOX = {...};
array<int, 16> PBOX_INV;

// 轮密钥（5 轮，每轮 16 位）
array<uint16_t, 5> ROUND_KEYS;
```

3.2 关键函数实现

加密函数

```
uint16_t encrypt(uint16_t plaintext) {
    uint16_t state = plaintext;
    for (int round = 0; round < 3; ++round) {
        state ^= ROUND_KEYS[round];
        state = sboxLayer(state);
        state = pboxLayer(state);
    }
    state ^= ROUND_KEYS[3];
    state = sboxLayer(state);
    state ^= ROUND_KEYS[4];
    return state;
}
```

差分攻击核心

```
for (int k2 = 0; k2 < 16; ++k2) {
    uint8_t u2 = SBOX_INV[c2 ^ k2];
    uint8_t u2Star = SBOX_INV[c2Star ^ k2];
    uint8_t diff2 = u2 ^ u2Star;

    for (int k4 = 0; k4 < 16; ++k4) {
        uint8_t u4 = SBOX_INV[c4 ^ k4];
        uint8_t u4Star = SBOX_INV[c4Star ^ k4];
        uint8_t diff4 = u4 ^ u4Star;

        if (diff2 == expectedDiff && diff4 == expectedDiff) {
            ++counts[k2][k4];
        }
    }
}
```

3.3 实验参数设置

- **样本数量**：6000 个明文对
- **输入差分**：0x0B00
- **目标 S 盒输出差分**：0x6
- **攻击目标**：第 4 轮密钥的第 2 和第 4 个 nibble（$K_4[2]$和 $K_4[4]$）

3.4 源代码

源码已上传至 Github：https://github.com/nhd-nana/Homework

四、实验结果

4.1 真实密钥

真实轮密钥 K4 的第 2、4 个 nibble(从高位开始计): 0x1 0xe

（程序随机生成，每次运行结果不同）

4.2 攻击结果示例

```
SPN 差分攻击演示
========================
真实轮密钥 K4 的第 2、4 个 nibble(从高位开始计): 0x1 0xe
=== 差分攻击 ===
样本数: 6000
目标差分: 0x6
候选密钥排序:
k2          k4          hits        score
0x1         0xe         296         0.049333
0x4         0xe         167         0.027833
0x1         0xb         159         0.026500
0x7         0xe         125         0.020833
0x1         0x8         120         0.020000
=== 重复统计 ===
试验次数: 100
成功率(真实密钥排第 1): 100.00%
平均候选密钥排名: 1.00
```

4.3 攻击成功率分析

样本数量	攻击成功率	平均候选密钥排名
100	64%	4.29
200	91%	1.83
400	96%	1.08
500	100%	1.00

五、结果分析

5.1 攻击有效性分析

1. **成功率**：当使用 500 个样本时，差分攻击能成功恢复正确的密钥 nibble，成功率在 99%以上。
2. **密钥区分度**：正确密钥的命中次数显著高于其他候选密钥，平均高出 2-3 倍。
3. **样本需求**：实验表明，约 400-500 个样本即可达到较高的攻击成功率。

5.2 影响因素分析

1. **差分特征概率**：选择的差分特征概率直接影响攻击效率，本实验使用的 0xB→6 差分概率约为 8/256≈3.1%。
2. **样本数量**：样本越多，正确的密钥特征越明显，但计算开销也相应增加。
3. **S 盒性质**：S 盒的非线性程度影响差分分布均匀性，理想的 S 盒应使最大差分概率尽可能小。

5.3 算法复杂度分析

- **时间复杂度**：O(N × 256)，其中 N 为样本数量
- **空间复杂度**：O(256)，用于存储密钥计数
- **攻击轮数**：可恢复最后 1 轮的部分密钥位

六、实验结论

1. **差分攻击有效性**：针对 4 轮 SPN 密码，差分攻击能在合理时间内恢复部分密钥位，证明了该攻击方法的实用性。
2. **安全启示**：
    - SPN 结构需要足够多的轮数来抵抗差分攻击
    - S 盒的设计应最小化最大差分概率
    - 建议轮数至少为 8 轮或以上
3. **攻击优化方向**：
    - 使用多重差分特征提高攻击效率
    - 结合密钥编排算法恢复完整密钥
    - 优化样本选择策略降低噪声影响

七、思考题

1. **如何提高攻击效率？**
    - 选择更高概率的差分路径
    - 使用多个差分特征并行攻击
    - 采用提前终止机制
2. **如何防御差分攻击？**
    - 增加加密轮数
    - 使用差分概率均匀的 S 盒
    - 引入动态 S 盒机制
    - 添加随机掩码
3. **如何扩展到完整密钥恢复？**
    - 先恢复最后一轮部分密钥
    - 解密一层得到前一轮状态
    - 逐层向前恢复所有轮密钥